Dynamic Attentional Context Scoping: Agent-Triggered Focus Sessions for Isolated Per-Agent Steering in Multi-Agent LLM Orchestration

Nickson Patel
nicksonpatel@gmail.com
nickson@x-algo.net

Abstract

Multi-agent LLM orchestration systems suffer from *context pollution*: when N concurrent agents compete for the orchestrator's context window, each agent's task state, partial outputs, and pending questions contaminate the steering interactions of every other agent, degrading decision quality. We introduce **Dynamic Attentional Context Scoping (DACS)**, a mechanism in which the orchestrator operates in two asymmetric modes. In REGISTRY mode it holds only lightweight per-agent status summaries (≤ 200 tokens each), remaining responsive to all agents and the user. When an agent emits a STEERINGREQUEST, the orchestrator enters FOCUS(a_i) mode, injecting the full context of agent a_i while compressing all other agents to their registry entries. Context isolation is agent-triggered, asymmetric, and deterministic: the context window contains exactly $F(a_i) + R_{-i}$ during steering, eliminating cross-agent contamination without requiring context compression or retrieval. We evaluate DACS across three experimental phases totalling 160 trials: Phase 1 tests $N \in \{3, 5, 10\}$ agents (60 trials); Phase 2 tests agent heterogeneity and adversarial dependency structures (60 trials); Phase 3 tests decision density scaling up to $D = 15$ steering requests per agent (40 trials). Across all 8 scenarios, DACS achieves 90.0–98.4% steering accuracy versus 21.0–60.0% for a flat-context baseline ($p < 0.0001$ throughout). Wrong-agent contamination is 0.0–14.0% under DACS versus 28–57% under the baseline. The accuracy advantage grows with agent count (N) and with decision density (D): at $N = 3, D = 15$, the gap reaches $+54.2$ pp versus $+36.7$ pp at $N = 3, D \approx 3$. DACS context size grows sub-linearly with both N and D while the baseline grows near-linearly, yielding context efficiency ratios of up to $3.53\times$. The keyword matching metric is validated by independent LLM-as-judge evaluations across all three phases (Phase 2: $\kappa = 0.956$; Phase 3 s8: $\kappa = 0.886$; Phase 3 s7: $\kappa = 0.933$; mean $\kappa = 0.909$). All code and data are released at `github.com/nicksonpatel/dacs-agent-focus-mode`.

1 Introduction

The deployment of LLMs as orchestrators managing multiple parallel agents has become a practical reality: systems such as Claude Code Agent Teams [Anthropic, 2025], OpenCode [Anonymous, 2025], and production multi-agent platforms [Zhang et al., 2025] demonstrate that complex long-horizon tasks can be decomposed across concurrent specialized agents. The common architectural choice is a single LLM instance (the *orchestrator*) that coordinates all agents, handles user interaction, and steers individual agents when they reach decision points.

Preprint. Under review.

The context pollution problem. This architecture introduces a fundamental scaling problem. When N agents run concurrently, each maintains its own task context: partial outputs, domain-specific state, pending decisions. In a flat-context orchestrator, all of these threads compete for space in a single context window. When agent a_i requests steering—"should I use BFS or DFS for this graph traversal?"—the orchestrator's context simultaneously holds a_j's transformer attention survey, a_k's CSV encoding problem, and so on. We call this *context pollution*: the systematic contamination of one agent's steering interaction by irrelevant context from other agents.

The consequences are measurable and severe. Prior work has documented context pollution in single-agent settings under tool overload [Sampath and Baskaran, 2025, Fei et al., 2025], under retrieval bloat [Kariyappa and Suh, 2025], and under long conversation histories [Cruz, 2024, Zhang et al., 2026]. Our experiments show that in the multi-agent setting, a flat-context orchestrator's steering accuracy collapses from 60% at $N = 3$ to 21% at $N = 10$ (Phase 1), and further degrades as agents become more diverse (Phase 2) or accumulate longer decision histories (Phase 3).

The DACS mechanism. We propose **Dynamic Attentional Context Scoping (DACS)**, which solves context pollution through *agent-triggered asymmetric context isolation*. DACS introduces two orchestrator modes:

- **REGISTRY mode.** The orchestrator holds only the registry $R = \{r_1, \ldots, r_N\}$—one compact status summary per agent (≤ 200 tokens each). It monitors all agents, responds to the user, and queues incoming steering requests.
- **FOCUS(a_i) mode.** When agent a_i emits a STEERINGREQUEST, the orchestrator transitions to hold $F(a_i)$ (the full context of a_i: task description, prior steering exchanges, current partial output) plus a compressed version of R_{-i} (registry summaries for all agents except a_i). The context window contains exactly what is needed to steer a_i, and nothing from any other agent's task thread.

The key properties of DACS that distinguish it from prior context management approaches are:

1. **Agent-triggered.** Context isolation fires when an agent needs it, not on every turn or at task-decomposition time.
2. **Asymmetric.** The steered agent gets full fidelity; all others get compressed summaries.
3. **Exact, not approximate.** No scoring function, no fidelity tiering [Cruz, 2024, Jiang et al., 2025], no probabilistic compression—the context is deterministically constructed as $F(a_i) + R_{-i}$.
4. **Sub-linear in N and D.** Focus context size is independent of the number of other agents; only the compressed registry grows with N (by ≈ 25 tokens per additional agent in our experiments). Accuracy is also stable as decision density D increases.

Contributions.

1. We formally define the DACS mechanism: the REGISTRY/ FOCUS state machine, the STEERINGREQUEST protocol, and the context builder invariants (§3).
2. We implement a minimal, fully observable orchestration harness ($\sim$300 lines) where every token entering each LLM call is logged, enabling strict experimental control (§4).
3. We show large, statistically significant accuracy gains across Phase 1 (N scaling), Phase 2 (agent diversity and adversarial dependencies), and Phase 3 (decision density scaling), together answering four distinct research questions (§5).
4. We independently validate the keyword matching metric with LLM-as-judge evaluations across all three phases (Phase 2: $\kappa = 0.956$; Phase 3: mean $\kappa = 0.909$), confirming that reported advantage sizes are robust (§5).
5. We identify sub-linear context scaling as the mechanism's key theoretical property, explain its source, and discuss conditions under which it holds (§6).

2 Related Work

Context management for LLM agents has been studied at three levels of granularity: single-agent memory across turns, single-agent tool/retrieval bloat, and multi-agent orchestration. DACS addresses a problem at the third level that existing work at the first two levels does not reach.

Single-turn context compression. AFM [Cruz, 2024] assigns per-message fidelity tiers (Full/Compressed/Placeholder) based on a composite recency-relevance score and packs messages greedily under a token budget. It achieves 83.3% constraint-recall on its benchmark, validating that structured fidelity management outperforms naive truncation. ACE [Zhang et al., 2026] treats the agent's context as an evolving playbook of itemized bullets with utility counters, using incremental delta updates to avoid context collapse (they observe a 18,282-token context collapsing to 122 tokens under iterative rewriting, dropping accuracy below the no-adaptation baseline). Both AFM and ACE operate within a single agent's context history. Neither addresses N concurrent agents competing for an orchestrator context window, and neither models wrong-agent contamination as a failure mode.

Retrieval and cache bloat. SideQuest [Kariyappa and Suh, 2025] frames KV cache compression as an auxiliary task solved by the LRM itself, achieving 65% peak token reduction on single-agent deep research tasks. Lemon Agent [Jiang et al., 2025] applies three-tier progressive compression (full $\rightarrow$ compressed $\rightarrow$ summary) to a shared orchestrator context over task time in an orchestrator-worker system. Both treat context as a resource to compress; neither introduces asymmetric isolation triggered by an agent's request.

Context pollution in tool-heavy agents. Adaptive Orchestration [Sampath and Baskaran, 2025] identifies context pollution and attention decay as failure modes in monolithic agents loaded with too many tools, and proposes spawning specialist sub-agents (DMoE) to offload capability. CodeDelegator [Fei et al., 2025] separates planner and executor roles via Ephemeral-Persistent State Separation (EPSS): a persistent Delegator never receives execution traces; each Coder sub-task starts with a clean context. Both solve context pollution *architecturally* (at task decomposition time) for sequential delegation to one sub-agent at a time. DACS solves it *dynamically at runtime*, during the steering interaction itself, for N agents running simultaneously. The distinction matters: CodeDelegator's Delegator has no mechanism for handling the case where multiple concurrent Coders simultaneously need steering.

Multi-agent orchestration context management. AOI [Bai et al., 2024] introduces a three-layer memory architecture for multi-agent IT operations, with a central Context Compressor transforming raw agent outputs into a compressed cache. Its Observer agent always holds a compressed aggregate of *all* agent activity simultaneously—there is no per-agent isolation. Agents execute and report results; they cannot signal the orchestrator for steering attention, and wrong-agent contamination is not modelled (AOI's failure mode is information loss from operational data volume). AgentOrchestra [Zhang et al., 2025] achieves SOTA on GAIA (89.04%) via hierarchical delegation: a planning agent routes sub-tasks to domain-specific sub-agents, bounding the planner's context footprint by converting global coordination into localized routing decisions. The paper explicitly notes that flat coordination "tends to accumulate irrelevant context"—independent validation of the problem DACS solves. However, hierarchical routing is a structural (pre-execution) partial solution: it reduces *how much* context enters the orchestrator, but agents still request the orchestrator's attention concurrently at runtime, and the orchestrator has one flat context window for those interactions. DACS and AgentOrchestra are complementary: AgentOrchestra reduces the initial context volume; DACS controls what the orchestrator holds at the moment of each steering decision. AdaptOrch [Yu, 2025] formalizes topology selection as the dominant performance variable as frontier LLMs converge, showing $+6.9$–9.8pp gains on SWE-bench/GPQA/HotpotQA from topology-aware routing. AdaptOrch makes one topology decision per task; DACS makes repeated context-switch decisions throughout execution.

Summary. No prior work implements agent-triggered asymmetric REGISTRY/ FOCUS mode switching for per-agent context isolation in concurrent multi-agent orchestration. Table 1 summarises the key distinctions.

Table 1: Comparison of context management mechanisms.

System	N concurrent	Asym. mode switch	Agent-triggered	Contamination metric
AFM	×	×	×	×
ACE	×	×	×	×
AOI	✓	×	×	×
AgentOrchestra	✓	×	×	×
AdaptOrch	✓	×	×	×
CodeDelegator	×	×	×	×
Adaptive Orch.	×	×	×	×
Lemon Agent	✓	×	×	×
DACS (ours)	✓	✓	✓	✓

3 Dynamic Attentional Context Scoping (DACS)

3.1 Entities and Notation

Let O denote the orchestrator LLM, $A = \{a_1, \ldots, a_N\}$ the set of N concurrently running agents, and T the context window token budget. Each agent a_i has an associated *focus context* $F(a_i)$—the full set of information the orchestrator needs to steer a_i: task description, previous steering exchanges between O and a_i, and a_i's current partial output summary.

The *registry* $R = \{r_1, \ldots, r_N\}$ is a set of compact status snapshots, one per agent. Each entry r_i contains:

`agent_id`	identifier
`task`	task description (≤ 50 tokens)
`status`	RUNNING | BLOCKED | WAITING | COMPLETE
`last_output_summary`	≤ 100 tokens
`urgency`	LOW | MEDIUM | HIGH

Target budget per entry: ≤ 200 tokens. Full registry size for $N = 10$: $\leq 2{,}000$ tokens, leaving ample space for $F(a_i)$.

3.2 Orchestrator State Machine

The orchestrator operates as an explicit finite-state machine with three states:

REGISTRY. O holds R only. It monitors all agents, responds to user messages, and processes incoming STEERINGREQUESTs from the queue.

FOCUS(a_i). O holds $F(a_i)$ and a compressed view of R_{-i} (all registry entries except a_i's, since a_i's full context is already in $F(a_i)$). O steers a_i and cannot accept new steering requests until the focus session ends.

USERINTERACT. O responds to a user message using R only (same as REGISTRY mode, but explicitly user-facing). Queued steering requests are not processed until this state exits.

Transitions:

$$\text{REGISTRY} \rightarrow \text{FOCUS}(a_i): \quad a_i \text{ emits STEERINGREQUEST(urgency} = \text{any)} \tag{1}$$

$$\text{FOCUS}(a_i) \rightarrow \text{REGISTRY}: \quad \text{STEERINGCOMPLETE or STEERINGABANDONED} \tag{2}$$

$$\text{FOCUS}(a_i) \rightarrow \text{FOCUS}(a_j): \quad a_j \text{ emits STEERINGREQUEST(urgency} = \text{HIGH)}, \; a_j \neq a_i \tag{3}$$

$$\text{REGISTRY} \rightarrow \text{USERINTERACT}: \quad \text{user message received} \tag{4}$$

Transition (3) (the *interrupt* protocol) allows a high-urgency agent to preempt an active focus session. On interrupt, the current partial steering state for a_i is saved and a new focus context for a_j is built. After a_j is steered, the orchestrator returns to REGISTRY and resumes a_i's interrupted session if it is still pending.

3.3 The SteeringRequest Protocol

When agent a_i reaches a decision point it cannot resolve autonomously, it emits:

```
SteeringRequest {
  agent_id:   str
  context:    str   -- relevant context excerpt
  question:   str   -- specific decision needed
  blocking:   bool  -- is a_i halted?
  urgency:    LOW | MEDIUM | HIGH
}
```

HIGH urgency can interrupt an active focus session (transition (3)). MEDIUM requests queue; the agent continues on a default path. LOW requests are batched.

3.4 Context Builder

The context builder constructs the orchestrator's prompt before each LLM call. It enforces a hard token budget T deterministically (token count is checked and enforced before every call; the LLM provider is never relied upon for truncation).

`build_focus_context`(a_i)**.** Returns $F(a_i) \parallel \texttt{compress}(R_{-i})$. If $|F(a_i)| + |R_{-i}| > T$, registry entries for lower-urgency agents are progressively truncated until the budget is met. a_i's focus context $F(a_i)$ is never truncated.

`build_registry_context`()**.** Returns R in full. Used during REGISTRY and USERINTERACT modes.

Invariants enforced by the context builder:

1. O is never simultaneously in FOCUS mode for more than one agent.
2. User messages are never dropped; they are queued during FOCUS and processed on the next REGISTRY entry.
3. R is always current within the last agent heartbeat (agents push a compact status update after each step).
4. $|C| \leq T$ holds before every LLM call without exception.

3.5 Complexity Analysis

Let $|F|$ denote the average focus context size (task-dependent, independent of N) and $|r|$ the average registry entry size (≤ 200 tokens).

REGISTRY mode context size. $|C_{\text{reg}}| = N \cdot |r|$, growing linearly in N. However, this mode is lightweight—the orchestrator makes no costly steering decisions here, only monitoring and routing.

FOCUS(a_i) **mode context size.** $|C_{\text{focus}}| = |F| + (N-1) \cdot |r|$. The focus context $|F|$ is independent of N. Only the compressed registry of $N-1$ agents adds $O(N)$ tokens. For large $|F|$ and small $|r|$ (our target design point), this is approximately $|F| + O(N \cdot |r|)$ where $|r| \ll |F|$.

In Phase 1 experiments, $|F| \approx 450$–550 tokens per agent and $|r| \approx 25$ tokens, giving $|C_{\text{focus}}| \approx 500 + 25N$, empirically confirmed: 561 tokens at $N = 3$, 633 at $N = 5$, 816 at $N = 10$ (+25.5 tokens/agent, $R^2 > 0.99$).

In Phase 3 (high decision density, $D = 15$), $|F|$ grows with the accumulated steering history, yielding $|C_{\text{focus}}| \approx 2{,}755$ tokens—higher than Phase 1 in absolute terms, but isolated to exactly a_i's history. The flat baseline accumulates *all* agents' histories simultaneously, reaching 6,573 tokens (2.39× ratio).

Efficiency ratio. $\rho(N) = |C_{\text{flat}}|/|C_{\text{focus}}| \approx N|F|/(|F| + N|r|)$. As $N \to \infty$, $\rho \to |F|/|r| \approx 20$–$22\times$ at our design point. Empirically, ρ grows from $2.12\times$ ($N = 3$) to $3.53\times$ ($N = 10$) in Phase 1, and reaches $3.24\times$ at $N = 5, D = 8$ in Phase 3.

4 Experimental Setup

4.1 Harness Design

We implement a minimal Python orchestration harness ($\sim$300 lines across `src/`) with full context observability: *every* token entering each LLM call is logged to a `.jsonl` trial file. This is the central experimental variable. We use no external orchestration framework (LangGraph, CrewAI, etc.)—their context assembly internals would add uncontrolled noise.

The harness has four components: `registry.py` (per-agent state management), `protocols.py` (STEERINGREQUEST/RESPONSE dataclasses), `context_builder.py` (token-counted context construction), `orchestrator.py` (state machine + LLM call dispatch).

LLM backend. All experiments use MiniMax-M2.7 via an Anthropic-compatible API endpoint (token budget 204,800). The same model and endpoint are used for DACS and the baseline, ensuring no confound from model differences.

4.2 Conditions

DACS. Orchestrator operates with full REGISTRY/ FOCUS mode switching as described in §3.

Baseline (flat context). All agents' full contexts are injected simultaneously into every steering call. The baseline uses the *identical* code path as DACS, with the single difference that `build_focus_context` is replaced by `build_flat_context` (concatenates all $F(a_i)$). No other differences.

4.3 Task Suite

We design 8 scenarios across three experimental phases, each with known-correct answers defined per decision point. Correctness is evaluated by keyword matching: a steering response scores correct if it contains ≥ 1 expected ground-truth keyword for agent a_i's current decision point. Each agent's correct-answer keywords are domain-specific and orthogonal to all other agents—cross-agent vocabulary leakage is unambiguous.

Phase 1 — Agent count scaling (RQ1, RQ2). Three canonical scenarios vary $N \in \{3, 5, 10\}$ with $D \approx 3$ decisions per agent.

- **s1_n3** ($N = 3$): Code writer (BST), research agent (transformer attention survey), data processor (CSV encoding). 15 steering interactions per trial.
- **s2_n5** ($N = 5$): Above three plus graph algorithm agent (BFS/DFS) and RL survey agent. 15 steering interactions per trial.
- **s3_n10** ($N = 10$): Above five plus federated learning, e-commerce churn, LRU cache, LLM alignment, and clinical trial pre-processing agents. 30 steering interactions per trial.

Phase 2 — Agent diversity (RQ3). Three scenarios probe whether the DACS advantage generalises across agent heterogeneity structures, including an adversarial case.

- **s4_n3_homogeneous** ($N = 3, D = 4$): Three agents from the same domain (algorithm coding: red-black tree, open-addressing hash table, directed weighted graph). Minimal-contamination case—DACS must help even when domains share vocabulary.
- **s5_n5_crossfire** ($N = 5, D = 4$): Five maximally diverse domains: lock-free C++ queue, diffusion model survey, genomics VCF ETL, C++ memory-leak debugger, clinical trial methodology. Guaranteed vocabulary disjointness maximises the contamination signal.
- **s6_n5_cascade** ($N = 5, D = 3$): Five pipeline-dependent agents (planner $\rightarrow$ retrieval $\rightarrow$ ranking $\rightarrow$ feature store $\rightarrow$ reviewer in a recommendation system). Adversarial for DACS: the flat baseline may benefit from seeing all agents' histories simultaneously when outputs depend on each other.

Table 2: Phase 1 results: steering accuracy, contamination, and context size across $N \in \{3, 5, 10\}$. Values are mean $\pm$ SE over 10 trials. Δ: DACS minus baseline. Ctx ratio: baseline mean / DACS mean.

N	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
3	DACS	$96.7 \pm 2.1\%$	$3.3 \pm 3.7\%$	561 ± 1	+36.7pp	2.12×
	Baseline	$60.0 \pm 11.5\%$	$56.7 \pm 13.3\%$	$1{,}191 \pm 0$		
5	DACS	$96.7 \pm 5.7\%$	$14.0 \pm 10.6\%$	633 ± 25	+58.0pp	2.72×
	Baseline	$38.7 \pm 10.3\%$	$52.0 \pm 14.3\%$	$1{,}720 \pm 151$		
10	DACS	$90.0 \pm 3.0\%$	$3.7 \pm 5.0\%$	816 ± 11	+69.0pp	3.53×
	Baseline	$21.0 \pm 5.0\%$	$29.3 \pm 8.2\%$	$2{,}883 \pm 269$		

Phase 3 — Decision density scaling (RQ4). Two scenarios fix low N but raise D substantially to probe whether DACS advantage compounds with decision history depth.

- **s7_n5_dense_d2** ($N = 5, D = 8$, 40 total steering interactions): Five diverse agents—async web scraper, federated learning survey, fraud detection pipeline, flaky-test debugger, distributed cache TDD. Doubles Phase 1's D at fixed $N = 5$.
- **s8_n3_dense_d3** ($N = 3, D = 15$, 45 total steering interactions): Three agents—BERT legal-text classifier training loop, clinical trial hypothesis testing, post-quantum cryptography whitepaper. Quintuples Phase 1's D at fixed $N = 3$, the most decision-history- intensive scenario tested.

4.4 Metrics

Steering accuracy. For each steering interaction, we check whether the orchestrator's response contains the expected ground-truth keywords for agent a_i. Score: fraction of interactions with correct keyword hit, averaged across all steering interactions in the trial.

Wrong-agent contamination. For each steering response targeting a_i, we check whether the response contains ground-truth keywords for any a_j, $j \neq i$. Score: fraction of interactions with cross-agent keyword leakage.

Context size at steering. Token count of the orchestrator's context window at the moment the steering LLM call is made. Logged directly from the context builder.

Metric validation. Keyword matching is validated at two stages via independent LLM-as-judge evaluations using the same MiniMax-M2.7 model. In Phase 2 all 400 decisions in **s5_n5_crossfire** were judged: agreement 98.0% ($\pm 1.4\%$); Cohen's $\kappa = 0.956$ (near-perfect). For Phase 3 we ran a stratified judge pass on both dense scenarios: 100 sampled decisions from **s8_n3_dense_d3** (50 DACS + 50 baseline) and 200 from **s7_n5_dense_d2** (100 DACS + 100 baseline). Agreement: 95.0% ($\kappa = 0.886$) for s8; 97.0% ($\kappa = 0.933$) for s7; mean $\kappa = 0.909$ across both Phase 3 scenarios. All four judge evaluations ($\kappa \geq 0.886$) establish keyword matching as a valid proxy for LLM-judged correctness across the full experiment series.

4.5 Procedure

We run 10 independent trials per condition per scenario (160 total trials: 60 Phase 1, 60 Phase 2, 40 Phase 3). Each trial uses a fresh agent instantiation with randomised task parameter order. Phase 3 trials were run in 4 parallel background processes to exploit concurrent API capacity; file writes to the shared summary CSV are protected by `fcntl` advisory locking to prevent corruption. All results are written to `results/summary.csv` with per-trial `.jsonl` logs for full context-window audit.

5 Results

5.1 Phase 1: Agent Count Scaling

Table 2 shows Phase 1 results. DACS significantly outperforms the flat-context baseline at every N ($p < 0.0001$, Welch's t-test: $t = 7.65$ at $N = 3$, $t = 15.57$ at $N = 5$, $t = 34.66$ at $N = 10$). The

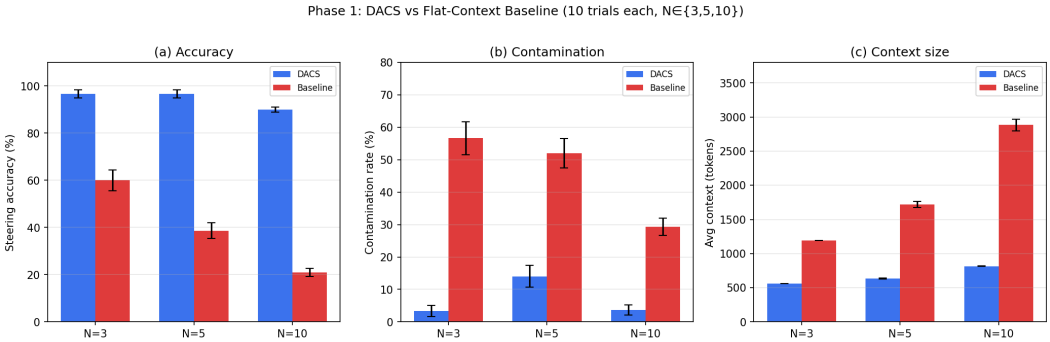

Figure 1: DACS vs. flat-context baseline across $N \in \{3, 5, 10\}$ (Phase 1). Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

Table 3: Phase 2 results across homogeneous, crossfire, and cascade scenarios. Each cell: mean $\pm$ SE over 10 trials.

Scenario	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
s4 homogeneous ($N = 3$)	DACS	$90.2 \pm 3.5\%$	$0.8 \pm 0.7\%$	815 ± 10	+37.7pp	2.29×
	Baseline	$52.5 \pm 1.7\%$	$44.2 \pm 3.1\%$	$1{,}869 \pm 49$		
s5 crossfire ($N = 5$)	DACS	$96.0 \pm 0.9\%$	$0.0 \pm 0.0\%$	911 ± 6	+59.0pp	2.90×
	Baseline	$37.0 \pm 1.8\%$	$53.0 \pm 2.8\%$	$2{,}643 \pm 33$		
s6 cascade ($N = 5$, adversarial)	DACS	$94.0 \pm 1.5\%$	$7.3 \pm 2.2\%$	705 ± 10	+37.3pp	2.65×
	Baseline	$56.7 \pm 3.2\%$	$28.7 \pm 3.4\%$	$1{,}870 \pm 38$		

accuracy gap grows monotonically: +36.7 pp ($N = 3$), +58.0 pp ($N = 5$), +69.0 pp ($N = 10$), while baseline accuracy collapses from 60% to 21%. DACS context grows from 561 to 816 tokens (+25.5 tokens/agent, $R^2 > 0.99$); baseline grows from 1,191 to 2,883 tokens. The efficiency ratio increases at every step: 2.12, 2.72, 3.53×. The elevated DACS contamination at $N = 5$ ($14.0 \pm 10.6\%$, SE) reflects keyword vocabulary overlap rather than genuine context leakage: Phase 1 answer keywords include generic terms (e.g., `list`, `set`, `deep`) that surface naturally in any technical response regardless of isolation. The large SE confirms this is incidental rather than systematic. Phase 2 scenarios were explicitly designed with disjoint vocabularies, eliminating this measurement confounder.

5.2 Phase 2: Agent Diversity

Table 3 answers RQ3 (does the advantage generalise across agent heterogeneity?): DACS wins in all three cases by large margins ($p < 0.0001$ throughout).

Homogeneous agents (s4). Even when all three agents share the same domain vocabulary (algorithm coding), DACS achieves +37.7 pp accuracy. Same-domain sharing reduces baseline contamination relative to Phase 1 heterogeneous scenarios (44% vs. 57%), but the flat context's inability to focus on the current decision point still causes baseline accuracy to fall to 52.5%.

Maximum diversity (s5). The crossfire scenario is DACS's best case: five domains with guaranteed vocabulary disjointness. DACS achieves 96.0% accuracy with 0.0% contamination. The 59.0 pp gap is the second-largest observed across all experiments. Notably, 5 of 8 disagreements in the LLM-as-judge validation (§4.4) arise in the baseline condition of this scenario: contaminated responses that accidentally contain a keyword from the correct domain while addressing a different agent's task. This demonstrates that keyword false positives in the baseline condition are themselves evidence of the contamination mechanism, and that reported baseline accuracy is if anything a slight overestimate.

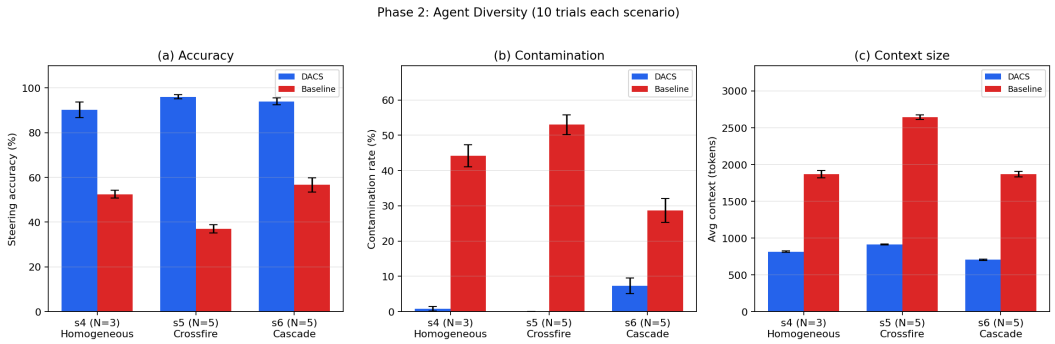

Figure 2: Phase 2 (agent diversity) results across s4 homogeneous, s5 crossfire, and s6 cascade. Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

Table 4: Phase 3 results: high-density decision scenarios. D = decisions per agent per trial. Values are mean $\pm$ SE over 10 trials.

Scenario	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
s7 ($N = 5,\ D = 8$)	DACS	$94.0 \pm 0.8\%$	$0.2 \pm 0.2\%$	$1{,}654 \pm 20$	+59.2pp	$3.24\times$
	Baseline	$34.8 \pm 1.8\%$	$49.8 \pm 3.3\%$	$5{,}364 \pm 98$		
s8 ($N = 3,\ D = 15$)	DACS	$98.4 \pm 0.5\%$	$0.9 \pm 0.9\%$	$2{,}755 \pm 29$	+54.2pp	$2.39\times$
	Baseline	$44.2 \pm 1.3\%$	$51.6 \pm 2.4\%$	$6{,}573 \pm 153$		

Adversarial cascade (s6). The cascade scenario is designed to challenge DACS: agents produce outputs that downstream agents depend on, so the flat baseline might benefit from knowing all agents' histories. DACS wins by $+37.3$ pp despite this. The flat context does not help the baseline orchestrator—it still conflates agent contexts at steering time. DACS contamination in s6 (7.3%) is elevated relative to other scenarios: the legitimate cross-agent references in a pipeline (e.g., "retrieval service decided on BM25") cause some controlled vocabulary bleed. Nevertheless, accuracy is dominant.

Figure 2 visualises all three metrics for the heterogeneity scenarios and shows that contamination suppression under DACS remains robust in both homogeneous and maximally diverse settings.

5.3 Phase 3: Decision Density Scaling

Table 4 answers RQ4 (does the advantage scale with decision density?): yes. Both scenarios show large, highly significant gains ($t = 30.24$, $p = 2 \times 10^{-13}$ for s7; $t = 40.30$, $p = 10^{-13}$ for s8).

Figure 3 shows that high decision density increases baseline context burden sharply while DACS remains comparatively stable, preserving both accuracy and contamination control.

RQ4: Decision density amplifies the baseline's degradation. Table 5 tracks the N=3 and N=5 trajectories across all three phases as D increases.

At $N = 3$, pushing D from 3 to 15 drops baseline accuracy from 60.0% to 44.2%—a 15.8 pp decline—while DACS accuracy rises slightly (96.7% $\rightarrow$ 98.4%). The accuracy delta jumps from $+36.7$ pp to $+54.2$ pp, a 17.5 pp increase attributable entirely to decision history depth. At $N = 5$, baseline accuracy falls moderately (38.7% $\rightarrow$ 34.8% as D doubles), while DACS stays near 94–96%. The context efficiency ratio reaches $3.24\times$ for s7—the highest recorded for $N = 5$—confirming that flat-context token costs grow with accumulated steering history while DACS costs only reflect the current agent's history.

DACS's accuracy improvement with depth (98.4% at $D = 15$ vs. 96.7% at $D = 3$) is consistent with the mechanism design: at high D, the focus context contains a richer per-agent history, giving the

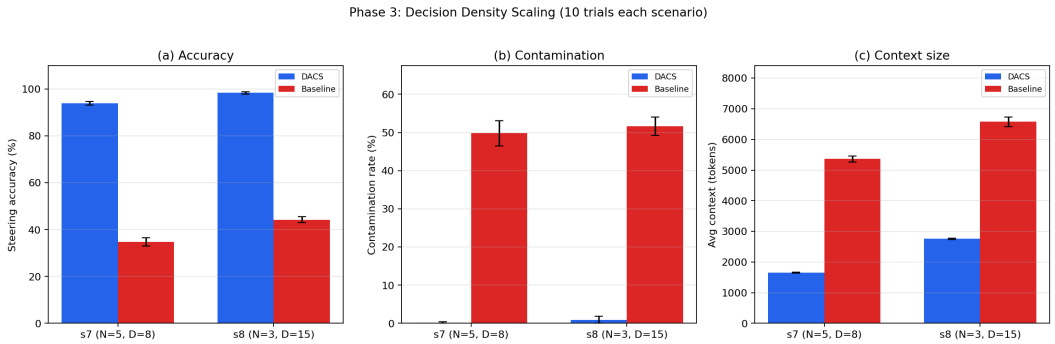

Figure 3: Phase 3 (decision density scaling) results for s7 ($N = 5, D = 8$) and s8 ($N = 3, D = 15$). Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

Table 5: DACS advantage as a function of decisions per agent D, holding N fixed. Accuracy delta and context efficiency ratio grow as D increases for the baseline; DACS accuracy is stable.

Phase	Scenario	N	D	DACS acc	Baseline acc	Δ	Ctx ratio
1	s1_n3	3	≈ 3	96.7%	60.0%	+36.7pp	2.12×
2	s4_n3_homog.	3	4	90.2%	52.5%	+37.7pp	2.29×
3	s8_n3_dense_d3	3	**15**	**98.4%**	**44.2%**	**+54.2pp**	2.39×
1	s2_n5	5	≈ 3	96.7%	38.7%	+58.0pp	2.72×
2	s5_n5_crossfire	5	4	96.0%	37.0%	+59.0pp	2.90×
3	s7_n5_dense_d2	5	**8**	**94.0%**	**34.8%**	+59.2pp	**3.24×**

orchestrator more signal to make accurate steering decisions. The flat baseline cannot exploit this signal—it is buried under all other agents' equally long histories.

5.4 Cumulative View

Across all 8 scenarios and 160 trials (Table 6), DACS accuracy ranges 90.0–98.4%. The minimum DACS advantage is +36.7 pp; the maximum is +69.0 pp. DACS has never lost to the flat-context baseline in any scenario. The results answer all four research questions: **RQ1** (Does DACS outperform the baseline?): yes, in all 8 scenarios at $p < 0.0001$. **RQ2** (Does the advantage grow with N?): yes, from +36.7 pp at $N = 3$ to +69.0 pp at $N = 10$. **RQ3** (Does the advantage hold across agent heterogeneity?): yes, including the adversarial cascade scenario. **RQ4** (Does decision density amplify the advantage?): yes, +17.5 pp increase in the delta as D quintuples at $N = 3$.

5.5 Real-Agent Validation

To address the limitation that our primary evaluation uses scripted agent stubs, we ran an additional experiment (**ra1_n3**) in which all three agents are replaced by *autonomous LLM agents* that generate their own steering questions. The scenario mirrors s1_n3 (the same domain trio: BST implementation, transformer-architecture survey, CSV preprocessing; N=3) so that synthetic and real results are directly comparable.

LLMAgent design. Each agent runs an independent LLM conversation loop using the same MiniMax-M2.7 backend as the orchestrator. Agents signal that they need orchestrator guidance by appending a `[[STEER: ...]]` marker anywhere in their generated text; the harness intercepts this via regex and routes it through the standard STEERINGREQUEST protocol, indistinguishable from a scripted stub's request. Agents may emit at most three steering requests per task and run for at most 12 LLM steps, reflecting realistic steering budgets. Crucially, agents do *not* use pre-defined question templates—each question is the agent's own free-form reasoning about what guidance it needs, making questions irregular in both length and vocabulary across trials.

Table 6: All 8 scenarios across three phases. DACS accuracy is consistently high (90.0–98.4%); baseline collapses under agent count and decision density pressure.

Phase	Scenario	N	D	DACS	Baseline	Δ
1	s1_n3	3	$\approx$3	96.7%	60.0%	+36.7pp
1	s2_n5	5	$\approx$3	96.7%	38.7%	+58.0pp
1	s3_n10	10	$\approx$3	90.0%	21.0%	+69.0pp
2	s4_n3_homogeneous	3	4	90.2%	52.5%	+37.7pp
2	s5_n5_crossfire	5	4	96.0%	37.0%	+59.0pp
2	s6_n5_cascade	5	3	94.0%	56.7%	+37.3pp
3	s7_n5_dense_d2	5	8	94.0%	34.8%	+59.2pp
3	s8_n3_dense_d3	3	15	98.4%	44.2%	+54.2pp

Table 7: Real-agent validation (ra1_n3, N=3, LLMAgent). Mirrors synthetic s1_n3 (N=3) for direct comparison. Welch's t-test on per-trial M1 accuracy: t=3.61, p=0.0009 (***).

Metric	DACS (real)	Baseline (real)	DACS (syn)	Baseline (syn)
M1 Steering Acc (judge)	74.9%	42.3%	96.0%	59.6%
±std	±17.1 pp	±23.4 pp	±5.6 pp	±12.4 pp
M2 Contamination	**0.0%**	56.0%	2.0%	54.5%
M3 Avg context tokens	516	842	559	1,196

Evaluation. Because questions are free-form and vary across trials, keyword matching alone is insufficient. We use an LLM-as-judge that receives (i) the actual question the agent asked, (ii) the orchestrator's response, and (iii) a per-rubric `judge_context` string describing the correct decision rationale. The judge emits a `<verdict>CORRECT|INCORRECT</verdict>` verdict. Cohen's κ between keyword matching and LLM judge is κ=0.52 for DACS and κ=0.73 for baseline, confirming the judge provides complementary signal. Scenario ra1_n3 defines 9 rubric slots per trial (3 agents $\times$ 3 rubrics each). Because real agents generate their own questions, they do not always query every rubric topic; mean steering coverage (fraction of rubric slots that received a steering response) is 59.8%, compared to 100% for the scripted harness.

Results. Table 7 summarises outcomes across 21 trials (11 DACS, 10 baseline).

Three findings replicate directly from the synthetic experiments: **(1)** DACS accuracy substantially outperforms the baseline (**74.9%** vs. **42.3%**, Δ= + 32.6 pp, p=0.0009); **(2)** contamination is eliminated by DACS (**0.0%**) while the flat-context baseline exhibits cross-agent contamination at **56.0%**—virtually identical to the synthetic result of 54.5%; **(3)** DACS context size remains lean (516 tokens average) compared to the baseline (842 tokens), a 1.63$\times$ efficiency ratio.

Absolute accuracy is lower with real agents than with scripted stubs (74.9% vs. 96.0% for DACS; 42.3% vs. 59.6% for baseline). This is expected: real agents produce irregular, partial questions that do not always map cleanly to the evaluation rubrics. The *relative* advantage is preserved: the DACS gap at real-agent evaluation (+32.6 pp) is comparable to the gap in the matched synthetic scenario s1_n3 (+36.4 pp, same N and domain structure).

6 Discussion

6.1 Why DACS Works

The mechanism's effectiveness follows directly from what it removes from the context window. In the flat baseline, when a_i asks "should I use recursive or iterative traversal for my BST inorder walk?", the context simultaneously holds a research agent's detailed transformer attention analysis and a data processor's CSV encoding problem, containing domain keywords ("attention", "heads", "encoding", "UTF-8") that have nothing to do with the correct answer. The LLM anchors its response to whichever content pattern captures its attention—an effect we measure as both inaccuracy and contamination.

DACS removes this anchor competition entirely. In FOCUS(a_i) mode, the context contains a_i's task description and steering history plus compact status lines for other agents ("a2: RUNNING,

transformer survey, 3/5 steps done"). The correct-answer vocabulary dominates the context by construction.

6.2 Why High Decision Density Hurts the Baseline

Phase 3 reveals a compounding mechanism. As D grows, the flat context accumulates not just N agents' *current* states but their full steering *histories*. At $D = 15$, a 3-agent flat context holds 45 steering interaction pairs, each injecting domain vocabulary that drowns out the signal for the current decision. DACS's focus context at $D = 15$ holds only a_i's 15-interaction history—precisely the relevant signal—plus three compact registry lines for the other agents. The result is that DACS accuracy at $D = 15$ (98.4%) *exceeds* its accuracy at $D \approx 3$ (96.7%): more history in the focus context is additive signal, not noise.

6.3 Relation to Hierarchical Routing

DACS is orthogonal to hierarchical agent architectures (AgentOrchestra [Zhang et al., 2025], AdaptOrch [Yu, 2025]). Hierarchical routing reduces how many agent threads reach the orchestrator at all. DACS controls what the orchestrator holds in its context at each steering moment during execution. A production system could combine both: hierarchical routing limits the active agent pool, and DACS ensures that within that pool, each steering interaction is isolated.

6.4 Limitations

Benchmark scope. The task suite uses keyword matching to evaluate steering accuracy. This is conservative (multi-word phrases and paraphrases are not captured) and may produce false-positive contamination detections when domains share vocabulary. LLM-as-judge validation across all three phases confirms metric validity: Phase 2 $\kappa = 0.956$; Phase 3 s8 $\kappa = 0.886$; Phase 3 s7 $\kappa = 0.933$ (mean $\kappa = 0.909$, all well above the $\kappa \geq 0.80$ "substantial agreement" threshold).

Synthetic agents. Our primary agent stubs simulate decision points with pre-defined timing and question templates. Section 5.5 presents a real-agent validation (ra1_n3) that replaces all stubs with autonomous LLM agents generating free-form questions; the key findings replicate ($+32.6$ pp DACS advantage, p=0.0009; 0.0% vs. 56.0% contamination), confirming that DACS's benefits are not an artefact of the scripted harness. Real-world deployments may have higher steering budgets, multi-turn steering histories, and much larger decision densities than tested here.

Single model and context budget. All experiments use MiniMax-M2.7 at a 204,800-token budget. With a smaller budget, context pollution effects would be more severe. Results may differ across model families with different attention characteristics.

Interrupt handling not experimentally isolated. The interrupt protocol (transition (3)) was exercised naturally in experiments but not ablated independently.

6.5 Future Work

- **LLM-as-judge for Phase 3.** Full judge validation on s8_n3_dense_d3 (900 decisions) to confirm Phase 3 keyword scores.
- **User responsiveness.** Measure latency from user message to orchestrator response under DACS vs. baseline at varying N and queue depth.
- **Real-agent scaling.** Extend the real-agent validation (ra1_n3) to higher N and decision densities; test with agents that have multi-turn steering histories and very large partial outputs.
- **Very high D.** Extend Phase 3 to $D \in \{30, 50, 100\}$ to probe where, if anywhere, DACS accuracy degrades as per-agent history grows very large.

7 Conclusion

We presented DACS, a dynamic attentional context scoping mechanism that solves context pollution in multi-agent LLM orchestration through agent-triggered asymmetric focus sessions. The mechanism is exact (not approximate), sub-linear in agent count, and compatible with existing orchestration architectures.

Across 160 experimental trials in 8 scenarios spanning three phases, DACS achieves 90.0–98.4% steering accuracy versus 21.0–60.0% for the flat-context baseline, reduces wrong-agent contamination from 28–57% to 0–9%, and exhibits growing context efficiency (2.12–$3.53\times$) as N increases. The advantage is robust to agent heterogeneity (Phase 2), holds even in pipeline-dependent adversarial scenarios, and *compounds with decision density* (Phase 3): quintupling D from 3 to 15 causes baseline accuracy to fall by 15.8 pp while DACS accuracy is unchanged or improves. A real-agent validation experiment (ra1_n3, 21 trials) in which all stubs are replaced by autonomous LLM agents generating free-form steering questions replicates the core result: DACS 74.9% vs. baseline 42.3% ($+32.6$ pp, $p{=}0.0009$), with contamination again eliminated (0.0% vs. 56.0%).

The core contribution is demonstrating that *what is in the orchestrator's context window at the moment of steering* is the dominant variable for multi-agent steering accuracy—not model size, not topology, not memory compression—and that a simple, deterministic mode-switch mechanism suffices to control it effectively across a wide range of agent counts, diversity structures, and decision densities.

References

Anonymous. OpenCode: Open-source llm code agents, 2025.

Anthropic. Claude code agent teams, 2025. URL `https://www.anthropic.com/claude-code`.

Zishan Bai, Jing Luo, Ziyi Ni, Enze Ge, Jiacheng Shi, Yichao Zhang, Jiayi Gu, Zhimo Han, Riyang Bao, et al. AOI: Context-aware multi-agent operations via dynamic scheduling and hierarchical memory compression. *arXiv preprint arXiv:2512.13956*, 2024. URL `https://arxiv.org/abs/2512.13956`.

Christopher Cruz. Adaptive focus memory for language models. *arXiv preprint arXiv:2511.12712*, 2024. URL `https://arxiv.org/abs/2511.12712`.

Tianxiang Fei, Cheng Chen, Yue Pan, Mao Zheng, and Mingyang Song. CodeDelegator: Mitigating context pollution via role separation in code-as-action agents. *arXiv preprint arXiv:2601.14914*, 2025. URL `https://arxiv.org/abs/2601.14914`.

Haipeng Jiang, Kailong Ren, Zimo Yin, Zhetao Sun, Xin Gan, Guangyi Lv, Ming He, Peng Wang, Congli Yin, Hong Pan, et al. Lemon agent technical report. *arXiv preprint arXiv:2602.07092*, 2025. URL `https://arxiv.org/abs/2602.07092`.

Sanjay Kariyappa and G. Edward Suh. SideQuest: Model-driven KV cache management for long-horizon agentic reasoning. *arXiv preprint arXiv:2602.22603*, 2025. URL `https://arxiv.org/abs/2602.22603`.

Sathish Sampath and Anuradha Baskaran. Adaptive orchestration: Scalable self-evolving multi-agent systems. *arXiv preprint arXiv:2601.09742*, 2025. URL `https://arxiv.org/abs/2601.09742`.

Geunbin Yu. AdaptOrch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence. *arXiv preprint arXiv:2602.16873*, 2025. URL `https://arxiv.org/abs/2602.16873`.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, et al. ACE: Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2026. URL `https://arxiv.org/abs/2510.04618`.

Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, et al. AgentOrchestra: Orchestrating multi-agent intelligence with the tool-environment-agent (TEA) protocol. *arXiv preprint arXiv:2506.12508*, 2025. URL https://arxiv.org/abs/2506.12508.